Introduction to Data Science

Modeling

Simon Munzert
Hertie School | GRAD-C11/E1339

Table of contents

1. Running models

2. Modeling with tidymodels

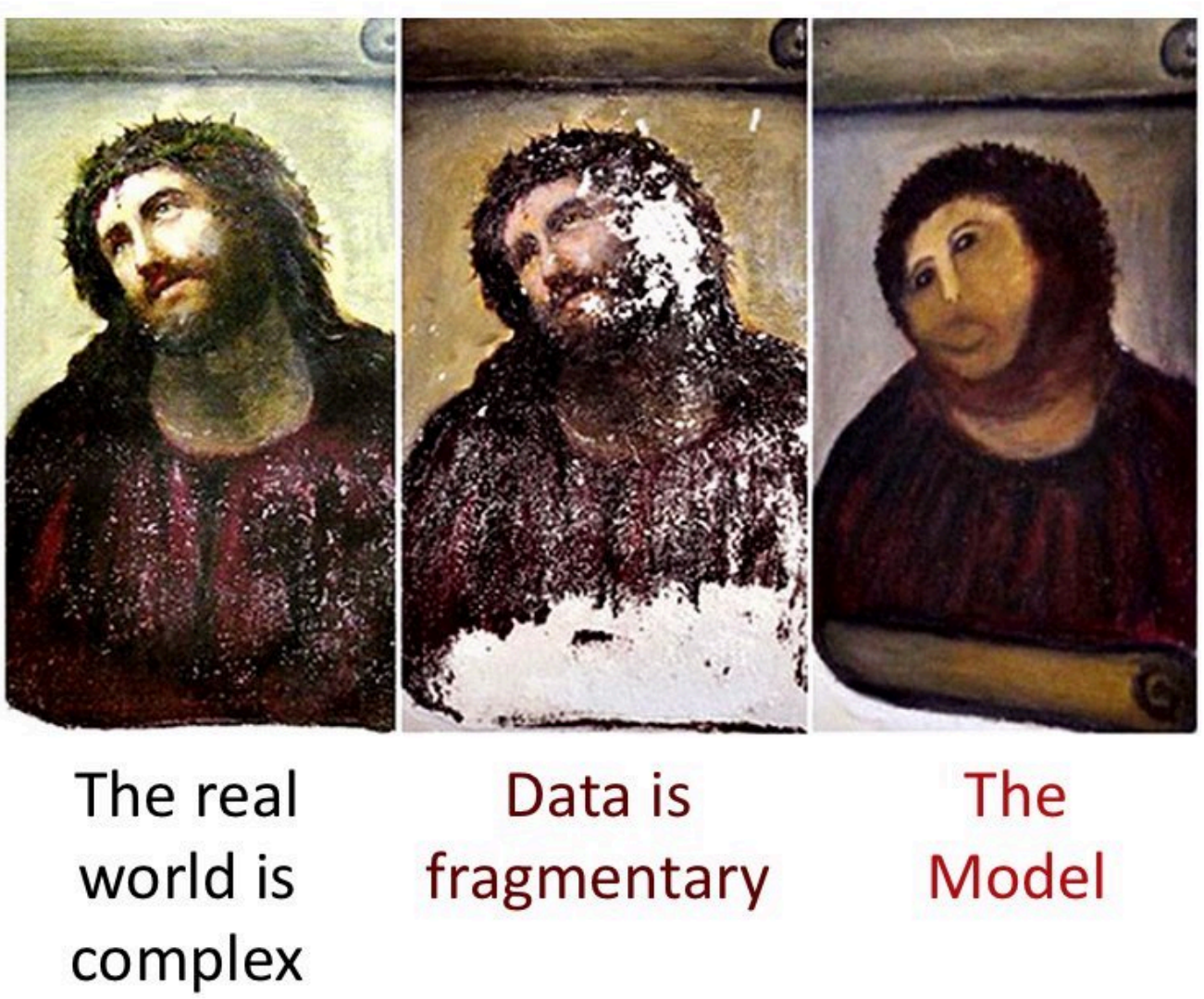
3. Processing estimation output

4. Reporting modeling results

5. Fitting many models

The modeling workflow

`Credit` David Hood



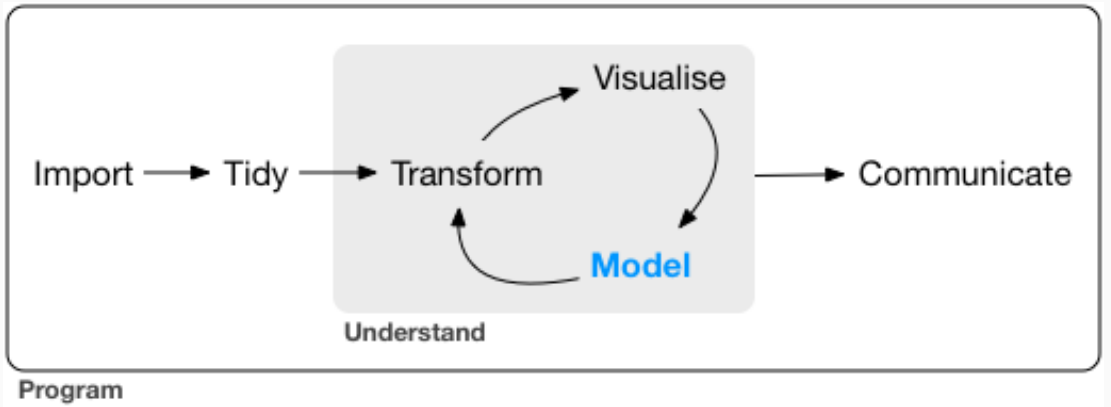
The modeling workflow

Why modeling?

- Modeling is at the heart of the data science workflow.
- We use models to explore, test, infer, predict based on data.
- The art and science of statistical modeling is vast.
- Today, we will focus on key steps of the workflow which are common in most modeling endeavors. We won't touch on theoretical/statistical backgrounds though and ignore workflow issues in particular areas, such as simulation-based Bayesian inference or cross-validation in ML.

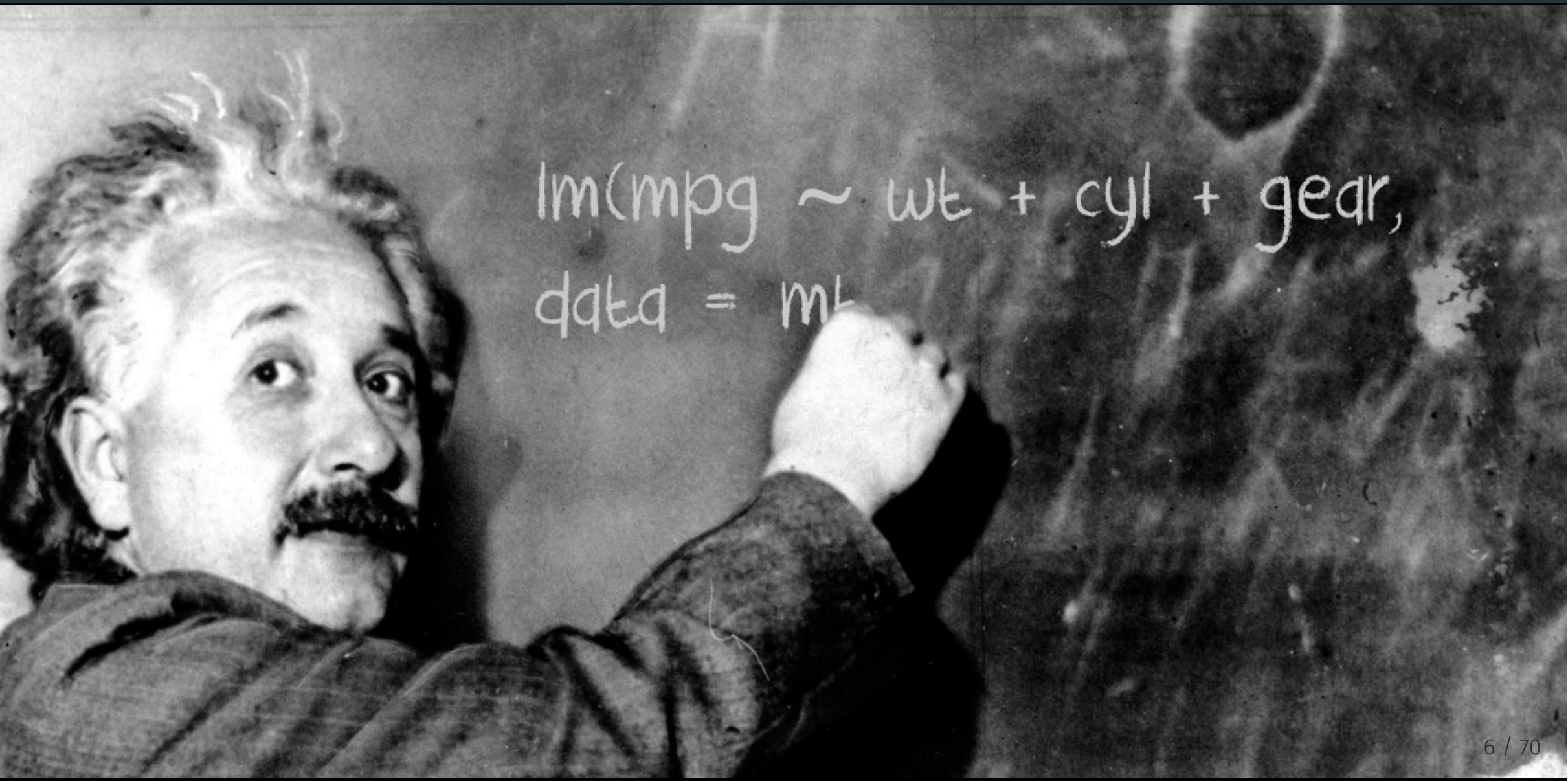
Steps of the workflow

1. **Choose** a modeling strategy
2. **Specify** the model (structural components / parameters)
3. **Run/implement** the model (estimation)
4. **Evaluate** the model output
5. **Present** the results



Running models

Running models



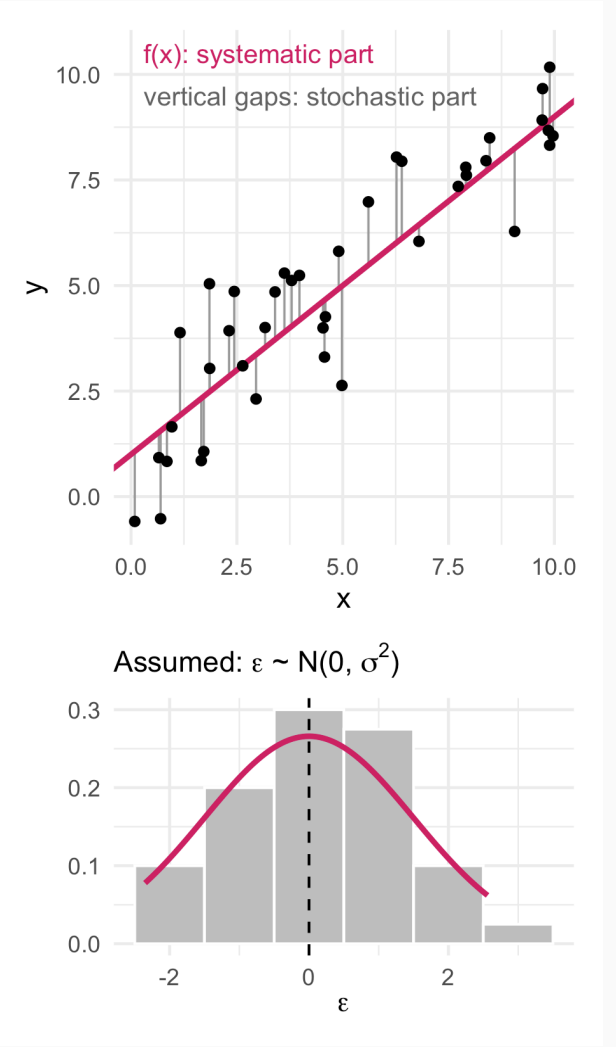
Model building

Systematic + stochastic components of models

- We often start by assuming a data-generating process of the form

$$Y = f(x) + \epsilon$$

- This decomposes the model into a **systematic** (structural, deterministic) part, $f(x)$, and a **stochastic** (random) part, ϵ.
- $f(x)$ captures what we think we can explain. ϵ collects everything else: unmeasured influences, measurement error, genuine randomness.
- We usually impose further assumptions on ϵ, e.g., $\epsilon \sim N(0, \sigma^2)$: errors are centered at zero (the model is right *on average*), have the same spread everywhere, and are independent of each other and of x. If they fail, estimates may still be fine, but uncertainty statements are off.
- To learn how variables relate to each other, we specify the systematic component, e.g. $f(x) = \beta_{0} + \beta_{1}x$, and "let the data speak" to estimate its parameters. But how do we express this in R?



A running example: flight delays

We model the **arrival delay** of flights from NYC airports in 2013 (`nycflights13`), using a random sample of 5,000 flights:

```
R> set.seed(42)
R> fl <- flights |>
+   drop_na(arr_delay, dep_delay) |>
+   slice_sample(n = 5000)
```

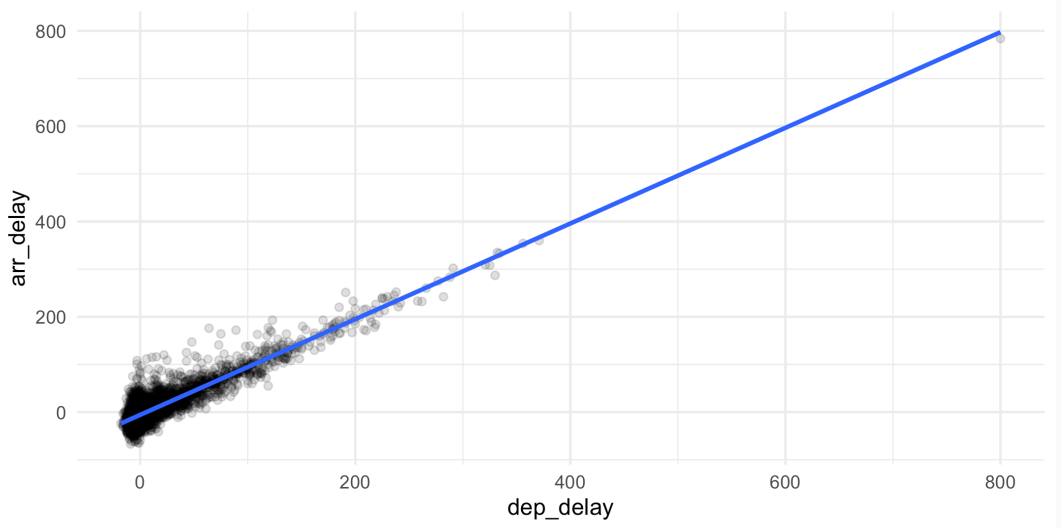
Candidate predictors: departure delay, distance, origin airport, carrier, ...

The simplest model: arrival delay as a linear function of departure delay,

$$\text{arrival delay} = \beta_0 + \beta_1 \, \text{departure delay} + \epsilon$$

The line is the systematic part. The vertical scatter around it is the stochastic part.

```
R> ggplot(fl, aes(dep_delay, arr_delay)) +
+   geom_point(alpha = .15) +
+   geom_smooth(method = "lm", se = FALSE) +
+   theme_minimal()
```



Full code, including some extra plots:

`materials/flight-delays-models.R`

Model formulas in R

R has a standardized way to specify the systematic part of a model: the `formula` class. The tilde separates outcome (left) and predictors (right):

```
R> m1 <- lm(arr_delay ~ dep_delay, data = fl)
R> coef(m1)
```

```
  (Intercept)   dep_delay
    -5.737477    1.003872
```

Formulas are objects. They can be stored, inspected, and built from strings:

```
R> fmla <- as.formula("arr_delay ~ distance")
R> class(fmla)
```

```
  [1] "formula"
```

```
R> m2 <- lm(fmla, data = fl)
```

Formula syntax cheat sheet

Formula	Meaning
`y ~ x1 + x2`	two predictors
`y ~ x1 - 1`	drop the intercept
`y ~ .`	all other columns as predictors
`y ~ x1 * x2`	main effects and interaction (`x1 + x2 + x1:x2`)
`y ~ x1:x2`	interaction only
`y ~ x + I(x^2)`	arithmetic inside `I()` (note: `y ~ x + x^2` does not work)
`y ~ log(x)`	function of a variable

Interactions

Question: Does the effect of a departure delay on the arrival delay differ by origin airport?

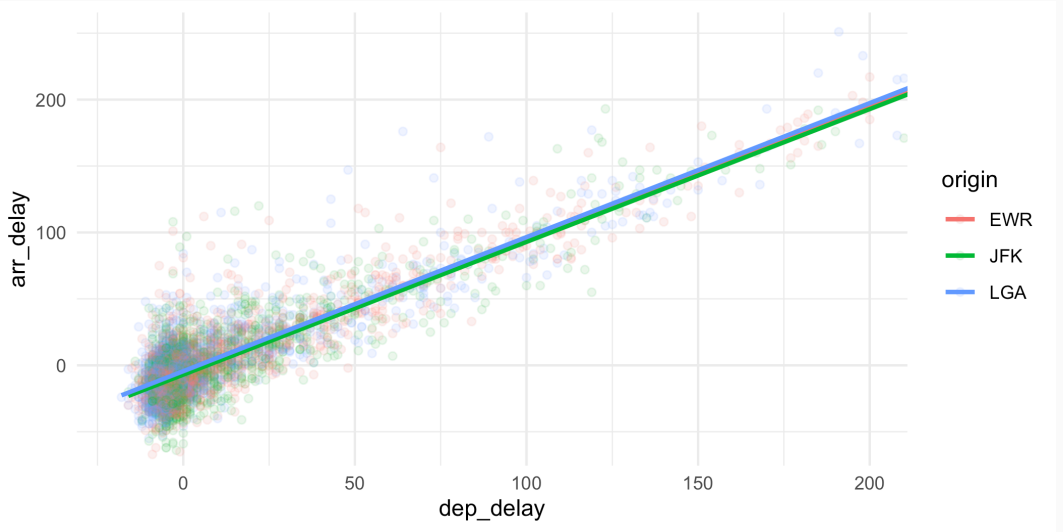
An *additive* model gives each airport its own intercept but a common slope. An *interaction* lets the slope vary by airport:

```
R> m_add <- lm(arr_delay ~ dep_delay + origin,
+              data = fl)
R> m_int <- lm(arr_delay ~ dep_delay * origin,
+              data = fl)
R> coef(m_int)[5:6] |> round(3)
```

```
  dep_delay:originJFK dep_delay:originLGA
               -0.007               0.003
```

`dep_delay:originJFK` is the difference in slope between JFK and EWR (the reference category). Here, the slopes are practically identical.

```
R> ggplot(fl, aes(dep_delay, arr_delay,
+                 color = origin)) +
+   geom_point(alpha = .1) +
+   geom_smooth(method = "lm", se = FALSE) +
+   coord_cartesian(xlim = c(-20, 200),
+                   ylim = c(-60, 250)) +
+   theme_minimal()
```

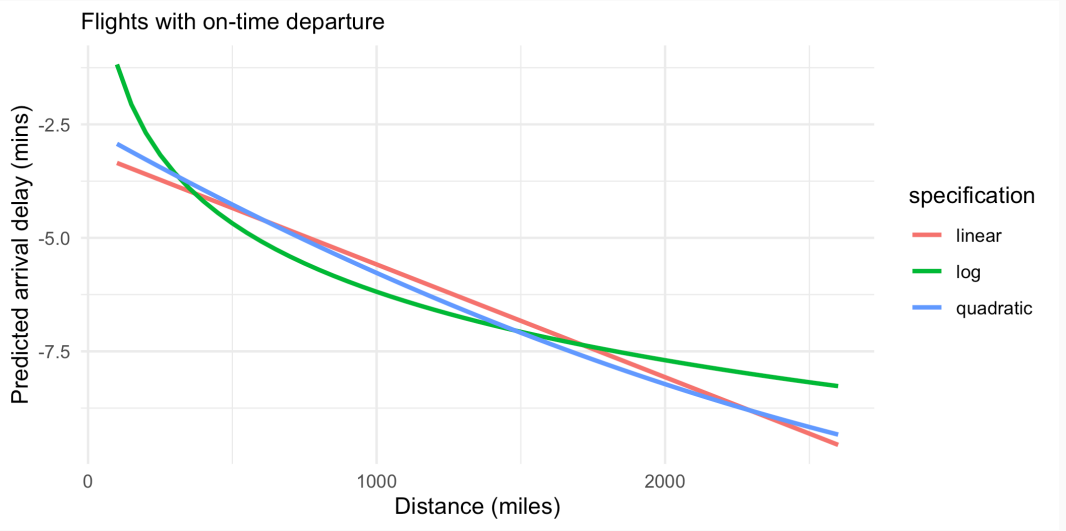


Variable transformations

Formulas don't evaluate arithmetic: in `arr_delay ~ distance^2`, the `^` is read as a formula operator, not as a square. Instead, wrap arithmetic in `I()` ("as is"), or use functions such as `log()`:

```
R> m_lin  <- lm(arr_delay ~ dep_delay + distance,
+               data = fl)
R> m_quad <- lm(arr_delay ~ dep_delay + distance +
+                 I(distance^2), data = fl)
R> m_log  <- lm(arr_delay ~ dep_delay + log(distance
+               data = fl)
```

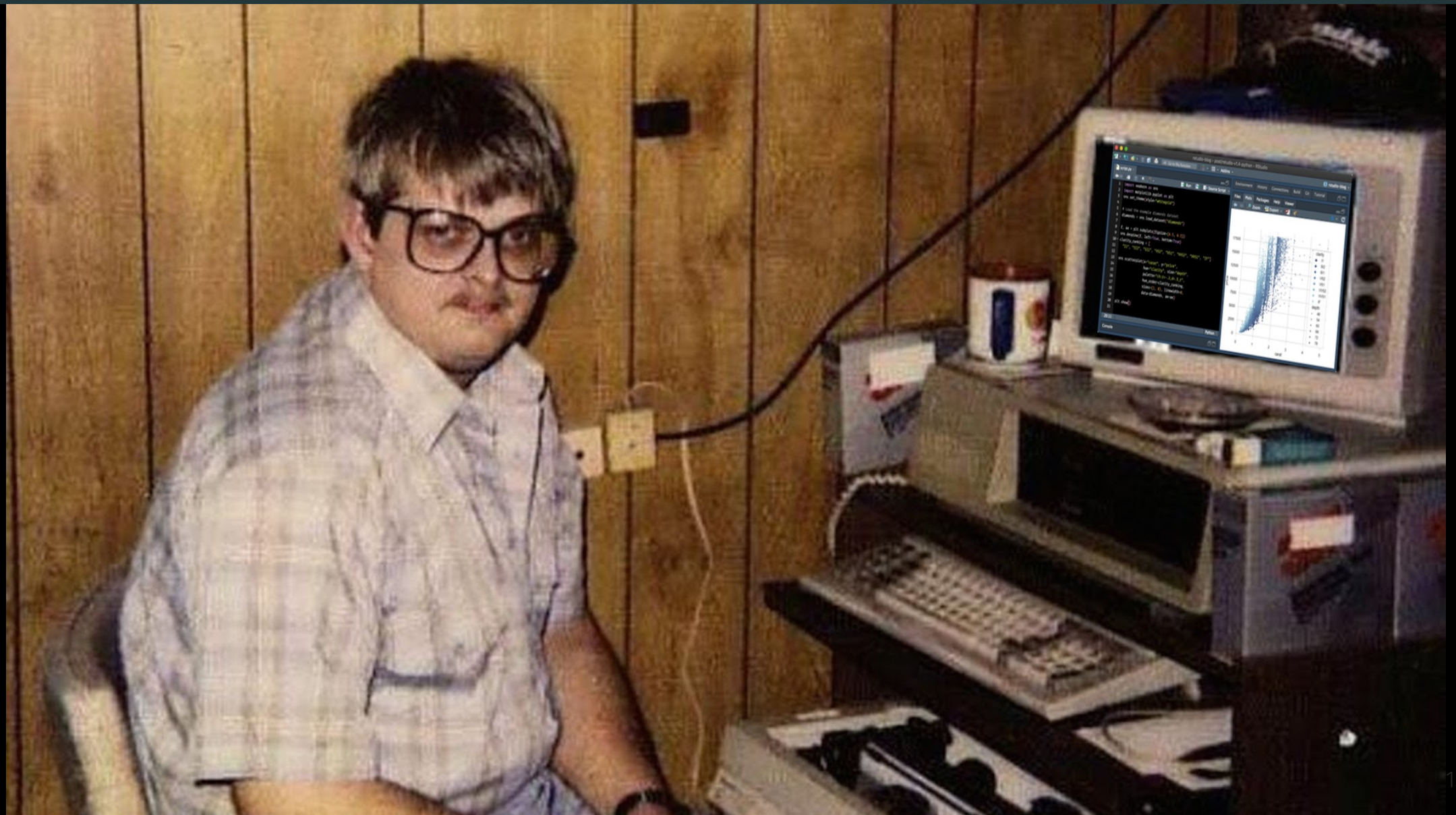
Alternatively, compute the variable beforehand (`mutate(distance2 = distance^2)`) and use it in the formula.



The plot shows what the three specifications imply for a flight that left on time. We could also plot the predictions for a flight that left 60 minutes late, or for each origin airport. The point is: the systematic part of the model is a *hypothesis* about how the data are generated. We can visualize it and check whether it makes sense.

Modeling with tidymodels

Modeling with tidymodels



Beyond the linear model

So far, we have used linear models. There is a multitude of model families extending them, e.g.:

- **Generalized linear models** [`stats::glm()`]: non-continuous responses (binary, counts)
- **Generalized additive models** [`mgcv::gam()`]: arbitrary smooth functions of predictors
- **Penalized models** [`glmnet::glmnet()`]: shrink coefficients to generalize better
- **Multilevel models** [`lme4::lmer()`]: nested data, varying intercepts/slopes
- **Trees and forests** [`rpart`, `ranger`], **boosting** [`xgboost`], **neural nets**, ...

For a glance at the universe from afar, see the CRAN Task Views.

Different modeling functions, different dialects

```
R> glm(late ~ dep_delay + distance, data = fl,
+     family = binomial)
R> mgcv::gam(arr_delay ~ s(dep_delay) + s(distance),
+           data = fl)
R> glmnet::glmnet(x = as.matrix(fl[, c("dep_delay",
+                 y = fl$arr_delay, alpha = 1)
R> lme4::lmer(arr_delay ~ dep_delay + (1 | carrier),
+            data = fl)
R> ranger::ranger(arr_delay ~ dep_delay + distance,
+                 data = fl, num.trees = 500)
```

Note the different inputs (formula vs. matrix), different argument names, different `predict()` behavior, different output objects.

Switching models means rewriting code. Routine steps (splitting data, cross-validation, tuning) get re-implemented by hand, with ample room for mistakes.

What is tidymodels?

tidymodels (developed by Max Kuhn and Julia Silge at Posit) is a collection of R packages for modeling and machine learning that share the design principles of the tidyverse: consistent interfaces, tidy data in and out, pipeable workflows.

Package	Purpose
`parsnip`	Specify models with one interface across engines
`broom`	Turn model output into tidy tibbles
`rsample`	Split and resample data
`recipes`	Preprocess and engineer features
`workflows`	Bundle preprocessing and model
`yardstick`	Measure model performance
`tune` / `dials`	Tune hyperparameters



Reading

- tidymodels: Get Started, articles 1–5
- Kuhn and Silge, *Tidy Modeling with R*

Four ideas behind tidymodels

1. Specify first, estimate later

A model is first *specified* (type, engine, mode) and only then *fitted*. Specifications are reusable, and parameters can be left open for tuning.

2. One interface for all models

The same code fits an `lm`, a Stan model, or a random forest. `predict()` always returns a tibble with one row per input row.

3. Never let the test data leak

Preprocessing that *learns* from data (centering, dummy coding, imputation) is estimated on training data only. The test set is touched at the very end.

4. Integrate the test workflow

Splitting, resampling, and tuning are all integrated in one workflow.

Note that `tidymodels` is **not a modeling method.** You still need to know what a model does. And tidymodels is built for *predictive* modeling; for *inferential* work, base R plus `broom` are fine.

We'll now look at the intuition behind each step and at a minimal example (check out the labs for details!).

```
R> library(tidymodels)
R> tidymodels_prefer() # resolve naming conflicts in favor of tidymodels
```

Step 1: Build a model

The idea

- A model specification answers three questions: **type** (linear regression, random forest, …), **engine** (which package does the estimation), **mode** (regression or classification).
- The specification has not seen data. Only `fit()` with a formula and data estimates it.
- Think `ggplot2`: declare what you want, then let the machinery produce it.
- Swapping the engine (`set_engine("stan")`, `set_engine("glmnet")`) is a one-line change.
- `broom` tidies the output (see later for details).

```
R> lm_mod <- linear_reg() |> set_engine("lm")
R> lm_fit <- lm_mod |>
+   fit(arr_delay ~ dep_delay + distance, data = fl)
R> tidy(lm_fit)
```

```
  # A tibble: 3 × 5
    term        estimate std.error statistic  p.value
    <chr>          <dbl>     <dbl>     <dbl>    <dbl>
  1 (Intercept) -3.10     0.447        -6.94 4.40e-12
  2 dep_delay    1.00     0.00649     154.   0
  3 distance    -0.00248  0.000340     -7.31 3.15e-13
```

```
R> predict(lm_fit, new_data = tibble(dep_delay = c(0
+                                    distance  = 1000
```

```
  # A tibble: 2 × 1
    .pred
    <dbl>
  1 -5.59
  2 54.6
```

Step 2: Preprocess the data

The idea

- Models rarely take raw data: categorical predictors need dummy coding, rare categories get lumped together, variables get transformed.
- Some of these steps *learn* from the data (which categories are rare, a mean for centering). If they learn from *all* data, test-set information leaks into training and evaluation becomes too optimistic.
- **Split first.** Then write a **recipe**: a list of preprocessing steps, estimated on the training data and applied unchanged to new data.
- A **workflow** bundles recipe and model, so that fitting and predicting always use both together.

We extend the model with more predictors: distance, origin airport, carrier, and scheduled hour of departure.

```
R> set.seed(123)
R> fl_split <- initial_split(fl)
R> train_data <- training(fl_split)
R>
R> fl_rec <- recipe(arr_delay ~ dep_delay + hour +
+                     distance + origin + carrier,
+                   data = train_data) |>
+   step_other(carrier, threshold = 0.05) |>
+   step_dummy(all_nominal_predictors())
R>
R> lm_wf <- workflow() |>
+   add_model(lm_mod) |>
+   add_recipe(fl_rec)
```

Step 3: Evaluate the model

The idea

- We want to know how a model performs on *new* data, not on the data it was fitted to.
- **Evaluating on the training data misleads:** every extra predictor improves the fit, even if it only picks up noise. Flexible models can memorize the training data.
- **Evaluating repeatedly on the test set misleads, too:** the test set becomes part of model selection.
- **Resampling:** split the training set into \(v\) folds, fit on \(v-1\) of them, score on the held-out fold, repeat, average. Use these estimates to compare models; keep the test set for one final check.
- Here we compare the one-predictor model with the extended one. RMSE is the typical prediction error in minutes.

```
R> set.seed(234)
R> folds <- vfold_cv(train_data, v = 5)
R>
R> simple_wf <- workflow() |>
+   add_model(lm_mod) |>
+   add_formula(arr_delay ~ dep_delay)
R>
R> simple_res <- fit_resamples(simple_wf, folds)
R> lm_res <- fit_resamples(lm_wf, folds)
R>
R> bind_rows(simple = collect_metrics(simple_res),
+            extended = collect_metrics(lm_res),
+            .id = "model") |>
+   filter(.metric == "rmse") |>
+   select(model, mean, std_err)
```

```
# A tibble: 2 × 3
  model     mean std_err
  <chr>    <dbl>   <dbl>
1 simple    18.0   0.155
2 extended  17.7   0.166
```

Step 4: Tune the model

The idea

- **Parameters** are learned from data (the coefficients). **Hyperparameters** are set *before* training and control model complexity.
- Example: the **lasso** penalizes the size of the coefficients. The larger the penalty, the more coefficients are set to exactly zero. How large should it be? Tune it.
- **Tuning:** mark the hyperparameter with `tune()`, try a grid of candidates on the resamples, pick the best, and fit *once* more on all training data. `last_fit()` then scores the test set, the first and only time it is used.
- Here the best penalty is small: with 5,000 flights and few predictors, there is little to regularize. The test-set RMSE matches the resampling estimate from the previous slide.

```
R> lasso_mod <- linear_reg(penalty = tune(),
+                          mixture = 1) |>
+   set_engine("glmnet")
R> lasso_wf <- lm_wf |> update_model(lasso_mod)
R>
R> grid <- tibble(penalty = 10^seq(-2, 2, by = 0.2))
R> lasso_res <- tune_grid(lasso_wf, folds,
+                         grid = grid)
R> best <- select_best(lasso_res, metric = "rmse")
R> best$penalty
```

```
  [1] 0.06309573
```

```
R> final_fit <- lasso_wf |>
+   finalize_workflow(best) |> last_fit(fl_split)
R> collect_metrics(final_fit) |>
+   filter(.metric == "rmse") |> pull(.estimate)
```

```
  [1] 17.37288
```

The whole workflow in one slide

```
# split
fl_split <- initial_split(fl)
train_data <- training(fl_split)
folds <- vfold_cv(train_data, v = 5)
# preprocess
fl_rec <- recipe(arr_delay ~ dep_delay + hour +
                   distance + origin + carrier,
                 data = train_data) |>
  step_other(carrier, threshold = 0.05) |>
  step_dummy(all_nominal_predictors())
# build (with open hyperparameter)
lasso_mod <- linear_reg(penalty = tune(), mixture = 1) |>
  set_engine("glmnet")
lasso_wf <- workflow() |>
  add_model(lasso_mod) |> add_recipe(fl_rec)
# tune, evaluate, last fit
grid <- tibble(penalty = 10^seq(-2, 2, by = 0.2))
lasso_res <- tune_grid(lasso_wf, folds, grid = grid)
best <- select_best(lasso_res, metric = "rmse")
final_fit <- lasso_wf |> finalize_workflow(best) |>
  last_fit(fl_split)
collect_metrics(final_fit)
```

Take it from there

- To try another model: replace `lasso_mod` (e.g., with `rand_forest()`). To try other features: edit the recipe. Everything downstream stays the same.
- Article 5 of Get Started runs this cycle on hotel bookings with a penalized logistic regression and a random forest, comparing them via ROC curves.
- What we didn't cover: `workflowsets` (many model/recipe combinations at once), iterative tuning (`tune_bayes()`), ensembles (`stacks`). See *Tidy Modeling with R*.

R vs. Python in predictive modeling

Why Python dominates in ML

- MDS folks: Next semester, you'll mostly work in Python (ML; DS&A).
- Python is the default for ML applications: `scikit-learn` for classical ML, `PyTorch` / `JAX` for deep learning, the LLM stack (`transformers`, …).
- Stronger on the engineering side: deployment, GPUs, integration into software products.

Where R is strong (or weak)

- Statistical modeling breadth: mixed and Bayesian models, GAMs, survey methods, causal inference.
- Exploration and reporting: `tidyverse`, `ggplot2`, Quarto.
- Classical ML is a wash: same C++ libraries under the hood. R clearly lags in deep learning and LLM tooling.

Share concepts

Idea	tidymodels	scikit-learn
Split, resample	`rsample`	`KFold`
Preprocess, no leakage	`recipes`	`Pipeline`
One model interface	`parsnip`	estimator API
Tune	`tune`	`GridSearchCV`
Evaluate	`yardstick`	`sklearn.metrics`

Both frameworks exist because workflow best practices (split first, resample to compare, tune on resamples, test once) are easy to get wrong by hand.

Processing estimation output

Processing estimation output



Why model processing?

When estimating a model, we usually estimate parameters (or simulate distributions thereof). There is, however, more that we can take away from the estimation, including:

```
summary(model_out)

Call:
lm(formula = arr_delay ~ distance + origin, data = flights)

Residuals:
    Min      1Q  Median      3Q     Max
 -89.04  -24.00  -11.83    7.26 1281.45

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) 13.4140488  0.1748144   76.73   <2e-16 ***
distance    -0.0040451  0.0001097  -36.87   <2e-16 ***
originJFK   -2.7042552  0.1887083  -14.33   <2e-16 ***
originLGA   -4.4561694  0.1935123  -23.03   <2e-16 ***
---
Sig. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 44.51 on 327342 degrees of freedom
(9430 observations deleted due to missingness)
Multiple R-squared: 0.005503, Adjusted R-squared: 0.005493
F-statistic: 603.7 on 3 and 327342 DF,  p-value: < 2.2e-16
```

Why model processing?

When estimating a model, we usually estimate parameters (or simulate distributions thereof). There is, however, more that we can take away from the estimation, including:

- **Estimated coefficients** and associated standard errors, T-statistics, p-values, confidence intervals

```
summary(model_out)

Call:
lm(formula = arr_delay ~ distance + origin, data = flights)

Residuals:
    Min      1Q  Median      3Q     Max
 -89.04  -24.00  -11.83    7.26 1281.45

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) 13.4140488  0.1748144   76.73   <2e-16 ***
distance    -0.0040451  0.0001097  -36.87   <2e-16 ***
originJFK   -2.7042552  0.1887083  -14.33   <2e-16 ***
originLGA   -4.4561694  0.1935123  -23.03   <2e-16 ***
---
Sig. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 44.51 on 327342 degrees of freedom
(9430 observations deleted due to missingness)
Multiple R-squared: 0.005503, Adjusted R-squared: 0.005493
F-statistic: 603.7 on 3 and 327342 DF,  p-value: < 2.2e-16
```

Why model processing?

When estimating a model, we usually estimate parameters (or simulate distributions thereof). There is, however, more that we can take away from the estimation, including:

- **Estimated coefficients** and associated standard errors, T-statistics, p-values, confidence intervals
- **Model summaries**, including goods of fit measures, information on model convergence, number of observations used

```
summary(model_out)

Call:
lm(formula = arr_delay ~ distance + origin, data = flights)

Residuals:
    Min      1Q  Median      3Q     Max
 -89.04  -24.00  -11.83    7.26 1281.45

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) 13.4140488  0.1748144   76.73   <2e-16 ***
distance    -0.0040451  0.0001097  -36.87   <2e-16 ***
originJFK   -2.7042552  0.1887083  -14.33   <2e-16 ***
originLGA   -4.4561694  0.1935123  -23.03   <2e-16 ***
---
Sig. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 44.51 on 327342 degrees of freedom
(9430 observations deleted due to missingness)
Multiple R-squared: 0.005503, Adjusted R-squared: 0.005493
F-statistic: 603.7 on 3 and 327342 DF,  p-value: < 2.2e-16
```

Why model processing?

When estimating a model, we usually estimate parameters (or simulate distributions thereof). There is, however, more that we can take away from the estimation, including:

- **Estimated coefficients** and associated standard errors, T-statistics, p-values, confidence intervals
- **Model summaries**, including goods of fit measures, information on model convergence, number of observations used
- **Observation-level information** that arises from the estimated model, such as fitted/predicted values, residuals, estimates of influence

```
summary(model_out)

Call:
lm(formula = arr_delay ~ distance + origin, data = flights)

Residuals:
    Min      1Q  Median      3Q     Max
-89.04  -24.00  -11.83    7.26 1281.45

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) 13.4140488  0.1748144   76.73   <2e-16 ***
distance    -0.0040451  0.0001097  -36.87   <2e-16 ***
originJFK   -2.7042552  0.1887083  -14.33   <2e-16 ***
originLGA   -4.4561694  0.1935123  -23.03   <2e-16 ***
---
Sig. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 44.51 on 327342 degrees of freedom
(9430 observations deleted due to missingness)
Multiple R-squared: 0.005503, Adjusted R-squared: 0.005493
F-statistic: 603.7 on 3 and 327342 DF,  p-value: < 2.2e-16
```

Processing estimation output in R

- Fitting a model returns an object of a certain model class (here: `lm`).
- Printing that object returns a quite minimalist set of information - just the input formula and coefficients.

```
R> model_out <- lm(arr_delay ~ distance + origin, data = flights)
R> class(model_out)
```

```
[1] "lm"
```

```
R> model_out
```

```
Call:
lm(formula = arr_delay ~ distance + origin, data = flights)

Coefficients:
(Intercept)     distance    originJFK    originLGA
  13.414049    -0.004045    -2.704255    -4.456169
```

Processing estimation output in R

- Fitting a model returns an object of a certain model class (here: `lm`).
- Printing that object returns a quite minimalist set of information - just the input formula and coefficients.
- The anatomy of the object is considerably more complex. It comes as a list of various components, including the coefficients, residuals, fitted values, and original model input.

```
R> names(model_out)
```

```
 [1] "coefficients"  "residuals"     "effects"       "rank"
 [5] "fitted.values" "assign"        "qr"            "df.residual"
 [9] "na.action"     "contrasts"     "xlevels"       "call"
[13] "terms"         "model"
```

Processing estimation output in R

- Fitting a model returns an object of a certain model class (here: `lm`).
- Printing that object returns a quite minimalist set of information - just the input formula and coefficients.
- The anatomy of the object is considerably more complex. It comes as a list of various components, including the coefficients, residuals, fitted values, and original model input.
- There's no way to print this list on the slide in full - it's just too long.

```
R> str(model_out)

 List of 14
  $ coefficients : Named num [1:4] 13.41405 -0.00405 -2.70426 -4.45617
   ..- attr(*, "names")= chr [1:4] "(Intercept)" "distance" "originJFK" "
  $ residuals    : Named num [1:327346] 3.25 16.77 26.7 -22.33 -30.88 ...
   ..- attr(*, "names")= chr [1:327346] "1" "2" "3" "4" ...
  $ effects      : Named num [1:327346] -3945.1 1579.9 204.2 1025 -30.8 .
   ..- attr(*, "names")= chr [1:327346] "(Intercept)" "distance" "originJ
  $ rank         : int 4
  $ fitted.values: Named num [1:327346] 7.75 3.23 6.3 4.33 5.88 ...
   ..- attr(*, "names")= chr [1:327346] "1" "2" "3" "4" ...
  $ assign       : int [1:4] 0 1 2 2
  $ qr           :List of 5
   ..$ qr   : num [1:327346, 1:4] -5.72e+02 1.75e-03 1.75e-03 1.75e-03 1.7
   .. ..- attr(*, "dimnames")=List of 2
   .. .. ..$ : chr [1:327346] "1" "2" "3" "4" ...
   .. .. ..$ : chr [1:4] "(Intercept)" "distance" "originJFK" "originLGA"
   .. ..- attr(*, "assign")= int [1:4] 0 1 2 2
   .. ..- attr(*, "contrasts")=List of 1
   .. .. ..$ origin: chr "contr.treatment"
   ..$ qraux: num [1:4] 1 1 1 1
```

Processing estimation output in R

- However, there are some high-level functions we can apply to do something useful with the model object, including:
  - `coef()` to extract the coefficients

```
R> coef(model_out)
```

```
(Intercept)    distance   originJFK   originLGA
13.414048769 -0.004045067 -2.704255237 -4.456169356
```

Processing estimation output in R

- However, there are some high-level functions we can apply to do something useful with the model object, including:
  - `coef()` to extract the coefficients
  - `fitted.values()` to extract the outcome values predicted by the model

```
R> coef(model_out)
```

```
  (Intercept)      distance    originJFK    originLGA
13.414048769 -0.004045067 -2.704255237 -4.456169356
```

```
R> fitted.values(model_out)[1:5]
```

```
        1        2        3        4        5
7.750955 3.230064 6.304715 4.334768 5.875538
```

Processing estimation output in R

- However, there are some high-level functions we can apply to do something useful with the model object, including:
  - `coef()` to extract the coefficients
  - `fitted.values()` to extract the outcome values predicted by the model
  - `residuals()` to extract the residuals

```
R> coef(model_out)
```

```
  (Intercept)      distance    originJFK    originLGA
13.414048769 -0.004045067 -2.704255237 -4.456169356
```

```
R> fitted.values(model_out)[1:5]
```

```
        1        2        3        4        5
7.750955 3.230064 6.304715 4.334768 5.875538
```

```
R> residuals(model_out)[1:5]
```

```
          1           2           3           4           5
  3.249045  16.769936  26.695285 -22.334768 -30.875538
```

Processing estimation output in R

- However, there are some high-level functions we can apply to do something useful with the model object, including:
  - `coef()` to extract the coefficients
  - `fitted.values()` to extract the outcome values predicted by the model
  - `residuals()` to extract the residuals
  - `model.matrix()` to extract the matrix of original input variables (predictors)

```
R> coef(model_out)
```

```
  (Intercept)     distance    originJFK    originLGA
13.414048769 -0.004045067 -2.704255237 -4.456169356
```

```
R> fitted.values(model_out)[1:5]
```

```
        1        2        3        4        5
7.750955 3.230064 6.304715 4.334768 5.875538
```

```
R> residuals(model_out)[1:5]
```

```
          1           2           3           4           5
  3.249045  16.769936  26.695285 -22.334768 -30.875538
```

```
R> model.matrix(model_out) %>% head(4)
```

```
  (Intercept) distance originJFK originLGA
1           1     1400         0         0
2           1     1416         0         1
3           1     1089         1         0
4           1     1576         1         0
```

Processing estimation output in R

- To learn more about the estimated model, we can apply the `summary()` function.
- The `summary` method is specific to the model class it is applied to (here: "`lm`"). To learn more, you'd have to call, e.g., `?summary.lm` or `?summary.glm`.

```
R> summary(model_out)
```

```
Call:
lm(formula = arr_delay ~ distance + origin, data = flights)

Residuals:
    Min      1Q  Median      3Q     Max
 -89.04  -24.00  -11.83    7.26 1281.45

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept) 13.4140488  0.1748144   76.73   <2e-16 ***
distance    -0.0040451  0.0001097  -36.87   <2e-16 ***
originJFK   -2.7042552  0.1887083  -14.33   <2e-16 ***
originLGA   -4.4561694  0.1935123  -23.03   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 44.51 on 327342 degrees of freedom
  (9430 observations deleted due to missingness)
Multiple R-squared:  0.005503,    Adjusted R-squared:  0.005493
F-statistic: 603.7 on 3 and 327342 DF,  p-value: < 2.2e-16
```

Processing estimation output in R

- To learn more about the estimated model, we can apply the `summary()` function.
- The `summary` method is specific to the model class it is applied to (here: "`lm`"). To learn more, you'd have to call, e.g., `?summary.lm` or `?summary.glm`.
- The function creates more than a printed summary in the console. It returns an object of class `summary.lm`, which can be further dissected.

```
R> class(summary(model_out))
```

```
[1] "summary.lm"
```

Processing estimation output in R

- To learn more about the estimated model, we can apply the `summary()` function.
- The `summary` method is specific to the model class it is applied to (here: "`lm`"). To learn more, you'd have to call, e.g., `?summary.lm` or `?summary.glm`.
- The function creates more than a printed summary in the console. It returns an object of class `summary.lm`, which can be further dissected.
- Again, there's no way to print this list on the slide in full - it's just too long.

```
R> str(summary(model_out))

  List of 12
   $ call         : language lm(formula = arr_delay ~ distance + origin, da
   $ terms        :Classes 'terms', 'formula'  language arr_delay ~ distanc
    .. ..- attr(*, "variables")= language list(arr_delay, distance, origin
    .. ..- attr(*, "factors")= int [1:3, 1:2] 0 1 0 0 0 1
    .. .. ..- attr(*, "dimnames")=List of 2
    .. .. .. ..$ : chr [1:3] "arr_delay" "distance" "origin"
    .. .. .. ..$ : chr [1:2] "distance" "origin"
    .. ..- attr(*, "term.labels")= chr [1:2] "distance" "origin"
    .. ..- attr(*, "order")= int [1:2] 1 1
    .. ..- attr(*, "intercept")= int 1
    .. ..- attr(*, "response")= int 1
    .. ..- attr(*, ".Environment")=<environment: R_GlobalEnv>
    .. ..- attr(*, "predvars")= language list(arr_delay, distance, origin)
    .. ..- attr(*, "dataClasses")= Named chr [1:3] "numeric" "numeric" "cha
    .. .. ..- attr(*, "names")= chr [1:3] "arr_delay" "distance" "origin"
   $ residuals    : Named num [1:327346] 3.25 16.77 26.7 -22.33 -30.88 ...
    ..- attr(*, "names")= chr [1:327346] "1" "2" "3" "4" ...
   $ coefficients : num [1:4, 1:4] 13.41405 -0.00405 -2.70426 -4.45617 0.1
    ..- attr(*, "dimnames")=List of 2
```

Dissecting model objects

The problem

"While model inputs usually require tidy inputs, such attention to detail doesn't carry over to model outputs. Outputs such as predictions and estimated coefficients aren't always tidy. This makes it more difficult to combine results from multiple models. For example, in R, the default representation of model coefficients is not tidy because it does not have an explicit variable that records the variable name for each estimate, they are instead recorded as row names. (...) This knocks you out of the flow of analysis and makes it harder to combine the results from multiple models. I'm not currently aware of any packages that resolve this problem."

Hadley Wickham, "Tidy Data"

The solution?

See next slide!

Processing estimation output with broom

`broom` is a suite of tools that summarizes key information about models. It takes the messy output of built-in functions in R, such as `lm` or `t.test`, and turns them into tidy `tibbles()` (think: dataframes). The output is not ready for publication but an important intermediary step that makes post-processing of estimation results more convenient. It is part of the tidyverse and tidymodels.



There are three key `broom` verbs that you need to learn.[1]

1. `tidy()`: Summarizes information about model components.
2. `glance()`: Reports information about the entire model.
3. `augment()`: Adds information about observations to a dataset.

[1] For a more detailed and comprehensive introduction, see the official documentation at https://broom.tidymodels.org/.

Tidy model objects with tidy()

`broom`'s `tidy()` function extracts the coefficient block (the model component) together with inferential statistics:

```
R> broom::tidy(model_out, conf.int = TRUE, conf.level = 0.95)
```

```
  # A tibble: 4 × 7
    term        estimate std.error statistic   p.value conf.low conf.high
    <chr>          <dbl>     <dbl>     <dbl>     <dbl>    <dbl>     <dbl>
  1 (Intercept) 13.4      0.175         76.7 0          13.1      13.8
  2 distance    -0.00405  0.000110     -36.9 5.53e-297 -0.00426  -0.00383
  3 originJFK   -2.70     0.189        -14.3 1.46e- 46 -3.07     -2.33
  4 originLGA   -4.46     0.194        -23.0 3.04e-117 -4.84     -4.08
```

Here, we also extract the upper and lower bounds on the 95% confidence intervals for the estimates. What makes the function so convenient is the fact that the output comes as a tidy `tibble` with useful variable names.

What exactly is extracted depends on the model type. You can learn more about the tidying function by typing `?tidy.[model class]`, e.g.: `?tidy.lm`.

Summarize model statistics with glance()

`broom`'s `glance()` function extracts summary statistics of the model and provides them in a single-row `tibble`:

```
R> broom::glance(model_out)
```

```
  # A tibble: 1 × 12
    r.squared adj.r.squared sigma statistic p.value    df    logLik     AIC    BIC
        <dbl>         <dbl> <dbl>     <dbl>   <dbl> <dbl>     <dbl>   <dbl>  <dbl>
  1   0.00550       0.00549  44.5      604.       0     3 -1706997.  3.41e6 3.41e6
  # ℹ 3 more variables: deviance <dbl>, df.residual <int>, nobs <int>
```

Augment data with model information with augment()

`broom`'s `augment()` function adds model information about each observation in a dataset, including, e.g.:

- predicted values (in the `.fitted` column)
- residuals (`.resid`)
- standard errors of fitted values (`.se.fit`; optional)

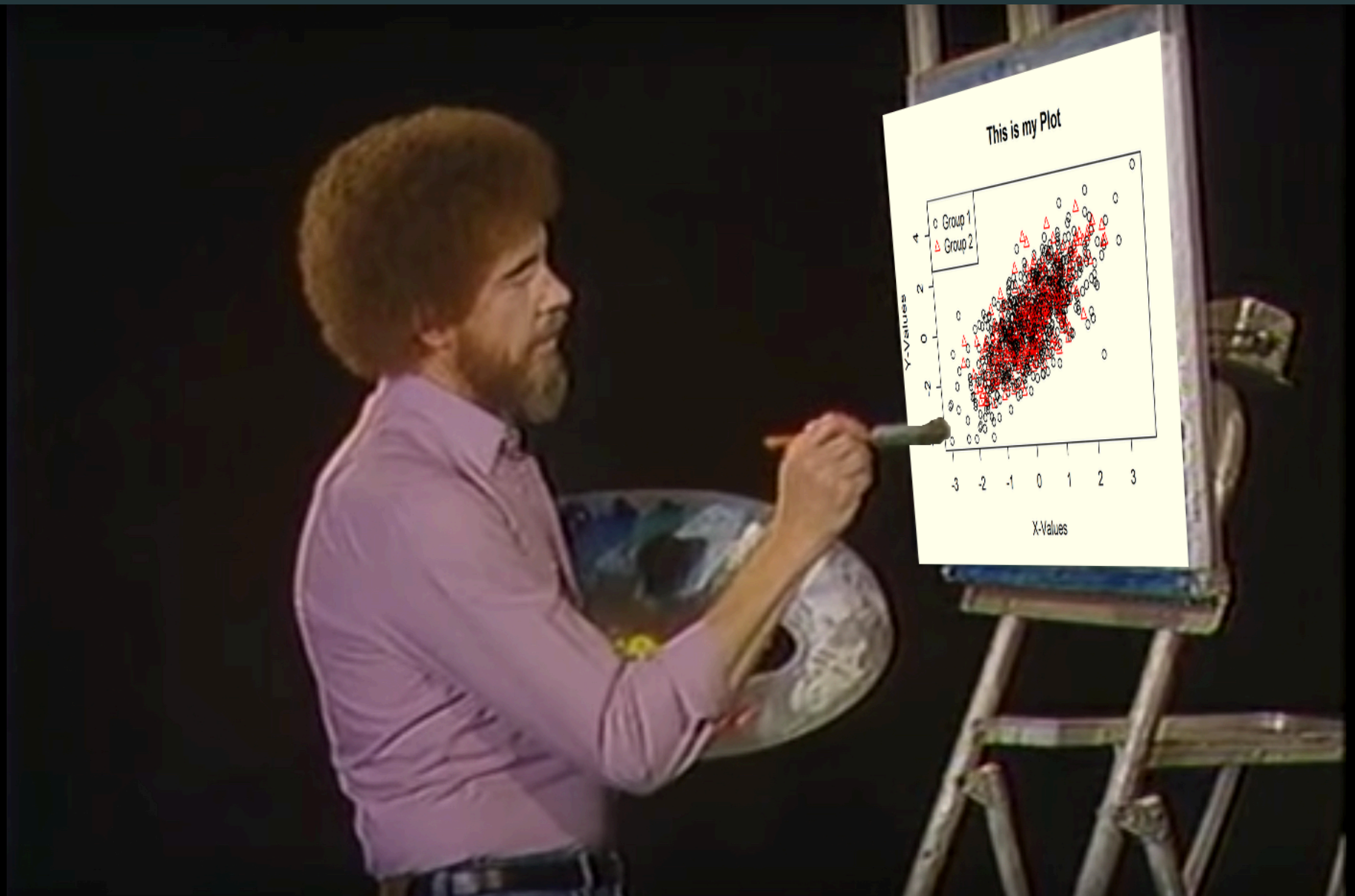
```
R> broom::augment(model_out, se_fit = TRUE) %>% head(3)
```

```
# A tibble: 3 × 11
  .rownames arr_delay distance origin .fitted .se.fit .resid       .hat .sigma
  <chr>         <dbl>    <dbl> <chr>    <dbl>   <dbl>  <dbl>      <dbl>  <dbl>
1 1                11     1400 EWR       7.75   0.135   3.25 0.00000922   44.5
2 2                20     1416 LGA       3.23   0.156  16.8  0.0000123    44.5
3 3                33     1089 JFK       6.30   0.136  26.7  0.00000938   44.5
# ℹ 2 more variables: .cooksd <dbl>, .std.resid <dbl>
```

It is also possible to pass on data that was not used during model fitting using the `newdata` argument. This requires that at least all predictor variable columns used to fit the model are present. Providing new data can be useful if one is interested to generate predictions for a test set.

Reporting modeling results

Reporting modeling results



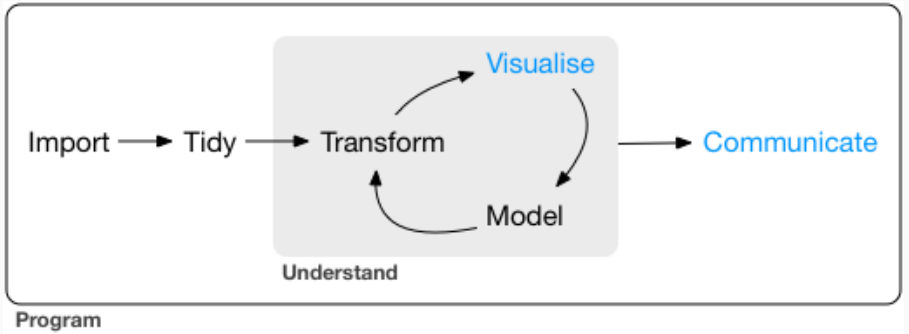
Reporting modeling results

Why good reporting is as important as good modeling

- Hardly anybody will read your code. Most stakeholders will not even be able to understand what you've done.
- Not all components of your model are equally relevant.
- Good communication of your model can save a lot of time and space.

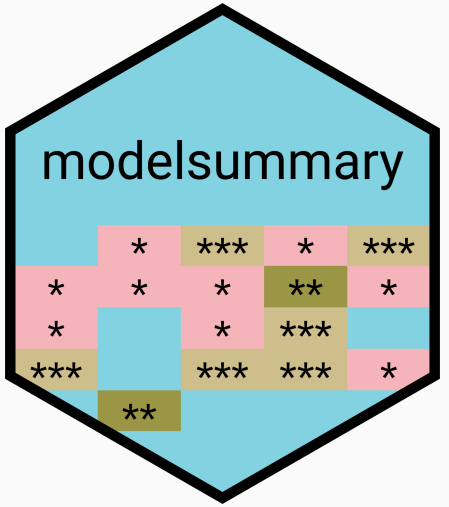
Making reporting part of the workflow

- Our vision of the data science workflow is to **automate as much as possible** in order to have time for the really important decisions.
- Since reporting results is usually at the end of the workflow, this step is affected by any change in the previous steps. Any manual work here hurts twice.
- Reporting and publishing results should be seen as part of the workflow. Even if we don't work with RMarkdown to write our reports, we want avoid copy-and-paste work into other software.



Summarizing models (and data) with modelsummary

`modelsummary` is a suite of tools to create table and plot summaries of models and data. It supports hundreds of model types out-of-the-box and the output can be saved to a wide variety of formats, including HTML, PDF, Text/Markdown, LaTeX, MS Word, JPG, and PNG.



There are three key `modelsummary` verbs that you need to learn.[1]

1. `modelsummary()`: Create regression tables with side-by-side models.
2. `modelplot()`: Create coefficient plots of model results.
3. `datasummary_*()`: Create data summaries such as cross-tabs or balance tables.

[1] There is more in `modelsummary` than what we can cover today. Have a glimpse at Vincent Arel-Bundock's page.

Creating regression tables with modelsummary()

- `modelsummary()` takes one or several models as input. Multiple models are provided as (optionally named) list.
- The extraction of information (estimates, standard errors, model summaries etc.) is taken care of by the function.
- On this slide you see how `modelsummary()` generates content ready to be rendered as HTML table. With the `output` argument, we can also ask the function to create `.tex`, `.rmd`, `.txt`, `.png`, and `.jpg`.
- Although the defaults are good, be sure to refine the table before publishing it.

```
R> modelsummary(list(model1_out, model2_out, model3_
+                output = "gt")
```

	(1)	(2)	(3)
(Intercept)	-5.899	10.829	-3.213
	(0.033)	(0.136)	(0.056)
dep_delay	1.019		1.018
	(0.001)		(0.001)
distance		-0.004	-0.003
		(0.000)	(0.000)
Num.Obs.	327346	327346	327346
R2	0.837	0.004	0.839
R2 Adj.	0.837	0.004	0.839
AIC	2822272.7	3414551.8	2818708.3
BIC	2822304.8	3414583.9	2818751.1
Log.Lik.	-1411133.361	-1707272.917	-1409350.174

Modifying regression tables

The `modelsummary()` function is extremely versatile. The defaults are good, but it will pay off to invest some time to learn the details, which are documented here.

In addition, it supports other table-making packages to further customize the appearance of tables. The details are documented here.

But I don't want to force anything on you. As always in R, there are several other excellent packages that help to create tables, including:

- `gtsummary` by Daniel Sjoberg
- `textreg` by Philip Leifeld
- `stargazer` by Marek Hlavac
- `sjPlot` by Daniel Lüdecke

In any case, do invest some time in learning the function's options and in actually producing readable and informative tables before publishing them. (1-3 hours per table are fine!)

```
R> modelsummary(
+   models,
+   output = "default",
+   fmt = 3,
+   estimate = "estimate",
+   statistic = "std.error",
+   vcov = NULL,
+   conf_level = 0.95,
+   stars = FALSE,
+   coef_map = NULL,
+   coef_omit = NULL,
+   coef_rename = NULL,
+   gof_map = NULL,
+   gof_omit = NULL,
+   group = term ~ model,
+   group_map = NULL,
+   add_rows = NULL,
+   align = NULL,
+   notes = NULL,
+   title = NULL,
+   escape = TRUE,
+   ...
```

modelsummary() tables: more examples

modelsummary package for R					
	Donations		Crimes (persons)		Crimes (property)
	OLS 1	Poisson 1	OLS 2	Poisson 2	OLS 3
Literacy (%)	-39.121	0.003***	3.680	-0.000***	-68.507***
	(37.052)	(0.000)	(46.552)	(0.000)	(18.029)
Priests/capita[1]	15.257		77.148**		-16.376
	(25.735)		(32.334)		(12.522)
Patents/capita		0.011***		0.001***	
		(0.000)		(0.000)	
Constant	7948.667***	8.241***	16259.384***	9.876***	11243.544***
	(2078.276)	(0.006)	(2611.140)	(0.003)	(1011.240)
Num.Obs.	86	86	86	86	86
R2	0.020		0.065		0.152
Adj.R2	-0.003		0.043		0.132
AIC	1740.8	274160.8	1780.0	257564.4	1616.9
BIC	1750.6	274168.2	1789.9	257571.7	1626.7
Log.Lik.	-866.392	-137077.401	-886.021	-128779.186	-804.441

[1] Very important variable.

* p < 0.1, ** p < 0.05, *** p < 0.01

The most important parameter is printed in red.

Table 1: modelsummary package for R

	Donations		Crimes (person)		Crimes (property)
	OLS 1	Poisson 1	OLS 2	Poisson 2	OLS 3
Literacy (%)	-39.121	0.003***	3.680	-0.000***	-68.507***
	(37.052)	(0.000)	(46.552)	(0.000)	(18.029)
Priests/capita	**15.257**		**77.148****		**-16.376**
	(25.735)		(32.334)		(12.522)
Patents/capita		0.011***		0.001***	
		(0.000)		(0.000)	
Constant	7948.667***	8.241***	16259.384***	9.876***	11243.544***
	(2078.276)	(0.006)	(2611.140)	(0.003)	(1011.240)
Num.Obs.	86	86	86	86	86
R2	0.020		0.065		0.152
Adj.R2	-0.003		0.043		0.132
AIC	1740.8	274160.8	1780.0	257564.4	1616.9
BIC	1750.6	274168.2	1789.9	257571.7	1626.7
Log.Lik.	-866.392	-137077.401	-886.021	-128779.186	-804.441

* p < 0.1, ** p < 0.05, *** p < 0.01

First custom note to contain text.

Second custom note with different content.

Tables vs. plots to communicate model results

The limits of tables

- Tables of coefficients work ok when models are linear and additive.
- They are good to communicate "precise" information.[1]
- They are, however, less informative for
  - non-linear relationships between \(x\) and \(y\) (\(x^2\), \(log(x)\), etc.),
  - interaction effects,
  - models for categorical data.

The promise of plots

- *Coefficient plots* can make it more straightforward to focus on two key features of estimated parameters: effect size and uncertainty.
- What's more, they make comparisons across effects and models much easier. We humans are visual animals.
- Other plots can go a long way to display other implications of models that are not visible from a tabular output, e.g., *predicted probability plots*, *marginal effects plots*, ...

[1] But sometimes give more precision than warranted. As a rule of thumb, never report more than three decimal points. In most cases, 0-2 is enough. Your estimates are less precise than that anyway.

Coefficient plots in the wild

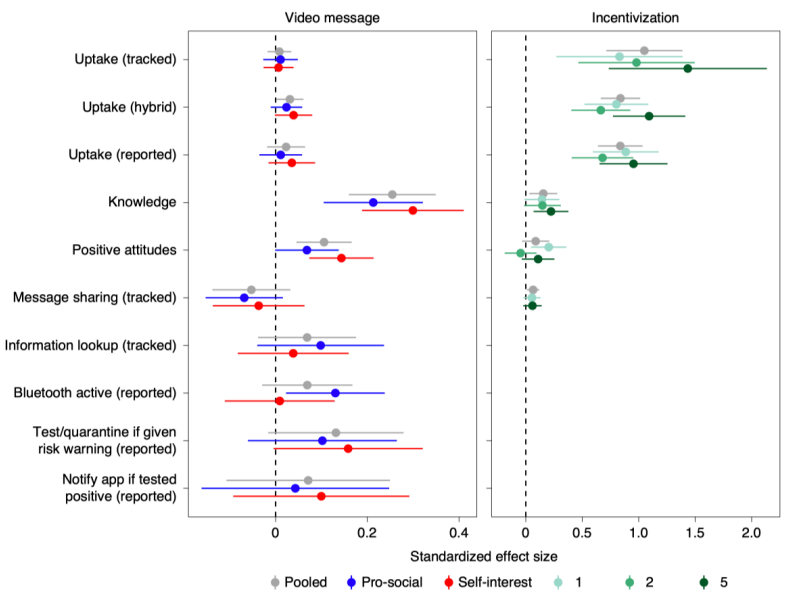
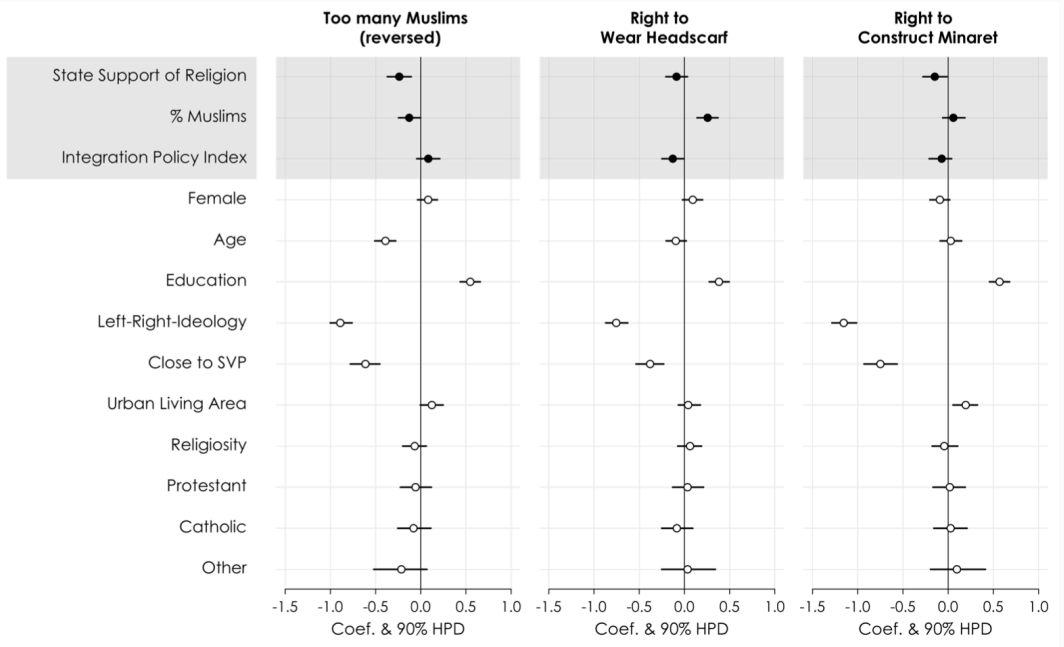


Fig. 3 | Effect of message and incentive treatments on uptake, knowledge, attitudes and behaviour. Each plot shows standardized ITT estimates with 95% CIs from fully saturated ordinary least squares regression models fit using the pre-registered LASSO covariate selection procedure. The video message sample comprises $n = 2{,}044$, 1,356 and 1,337 respondents for estimation of the pooled, pro-social and self-interest treatment effects, respectively. The incentive sample comprises $n = 1{,}015$, 513, 516 and 494 respondents for estimation of the pooled, €1, €2 and €5 treatment effects, respectively.

`Credit` Munzert et al. 2020

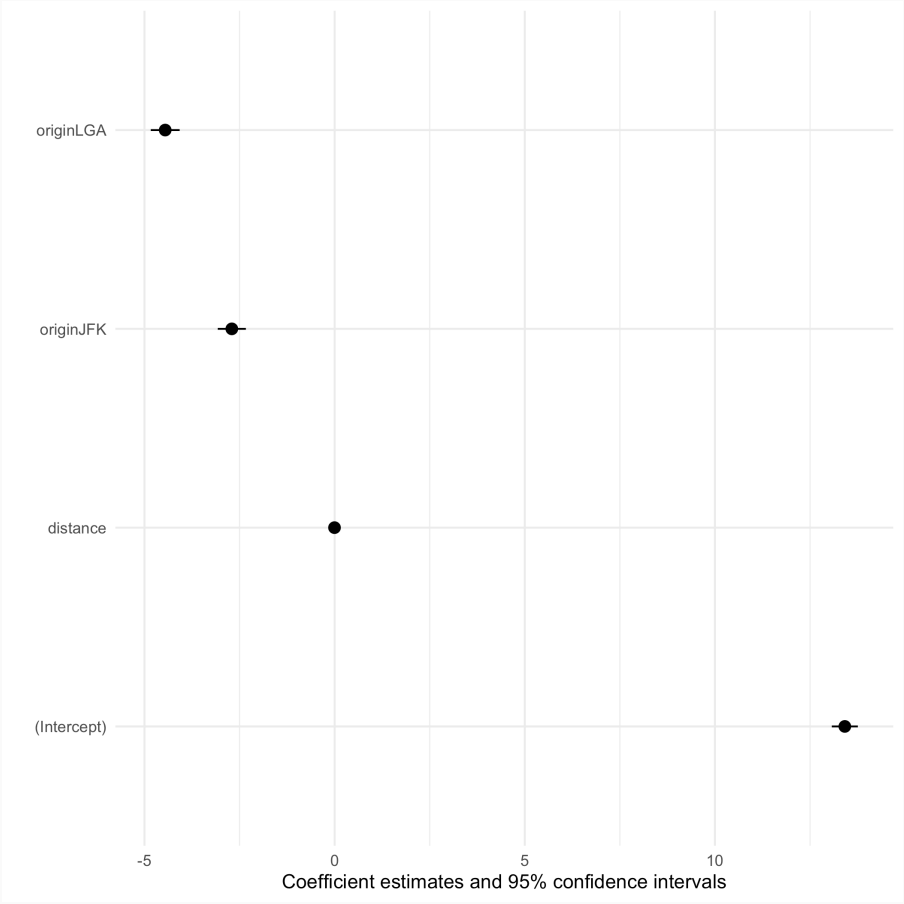


`Credit` Helbling and Traunmüller 2016

Creating coefficient plots with modelplot()

- `modelplot()` takes one or several models as input. Multiple models are provided as (optionally named) list.
- Again, the extraction of information (estimates, standard errors, model summaries etc.) is taken care of by the function.
- The graphs produced by `modelplot()` are simple `ggplot2` objects. They can be post-processed (and exported) accordingly.
- Although the defaults are good, be sure to refine the plot before publishing it.

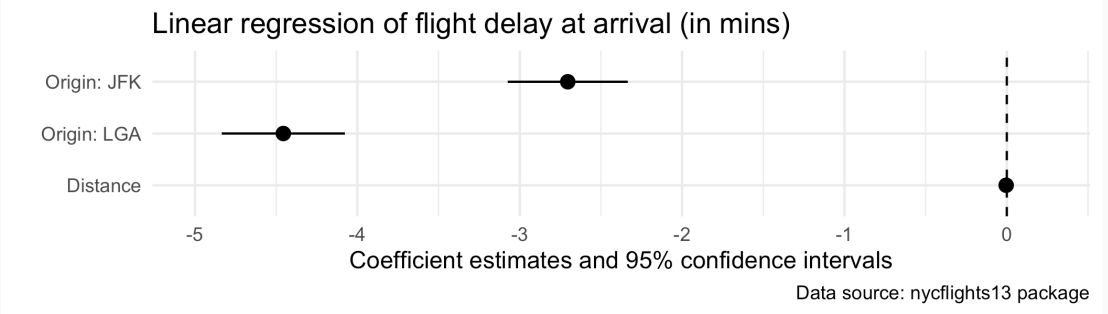
```
R> modelplot(model_out)
```



Modifying coefficient plots

- Here's one example for that.
- We provide intuitive variable names with `coef_map`.
- We drop the intercept with `coef_omit`.
- We make more layout adaptations with `ggplot2` functions that make the plot better readable.

```
R> cm <- c("distance" = "Distance",
+          "originLGA" = "Origin: LGA",
+          "originJFK" = "Origin: JFK")
R> modelplot(model_out,
+            coef_omit = "Interc",
+            coef_map = cm) +
+   xlim(-5, .25) +
+   geom_vline(xintercept = 0, linetype="dashed") +
+   labs(title = "Linear regression of flight delay at arrival (in mins)",
+        caption = "Data source: nycflights13 package") +
+   theme_minimal()
```



Working with interpretable effect sizes

Take care when plotting effect sizes

- One of the major perks of coefficient plots is the comparability of effects across coefficients.
- This can be, however, also one of the major problems of these kinds of plots.
- In order for them to make visual sense, the underlying covariates have to be inherently comparable. By showing slopes, the plot shows the effect of a unit change in each covariate on the outcome, but unit changes may not be comparable across variables.
- Also check out the documentation of the `effectsize` package for a more thorough discussion of the problem (and how to tackle it).

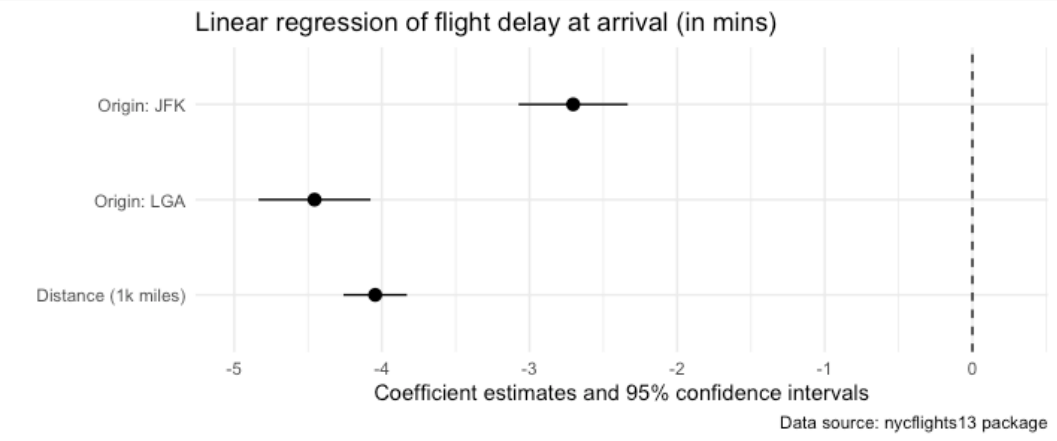
Addressing the issue

- There are several ways to address the problem, including:
  - Rescale variables to show intuitive unit changes in X (e.g., 1km instead of 1m)
  - Rescale to full scale (minimum to maximum) changes in X
  - Standardize variables to show standard deviation changes in X
- Note that any rescaling operation also affects how you interpret the coefficients (and we're only talking about the linear case!). Sometimes it also makes sense to standardize the response variable. In that case, the coefficients can be interpreted as the change in the response in standard deviations for a 1 unit change in the predictor (whatever that is).

Working with interpretable effect sizes (cont.)

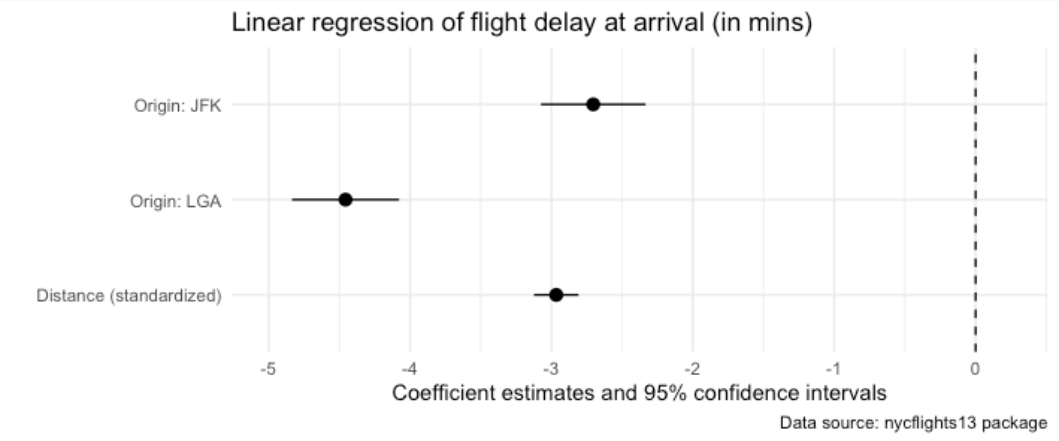
Rescaling (1 unit = 1000 miles)

```
R> # rescale continuous variable
R> flights$distance1kmiles <- flights$distance/1000
R> model_out_kmiles <- lm(arr_delay ~
+        distance1kmiles + origin, data = flights)
R>
R> # plot model (detailed fine-tuning not shown)
R> modelplot(model_out_kmiles)
```



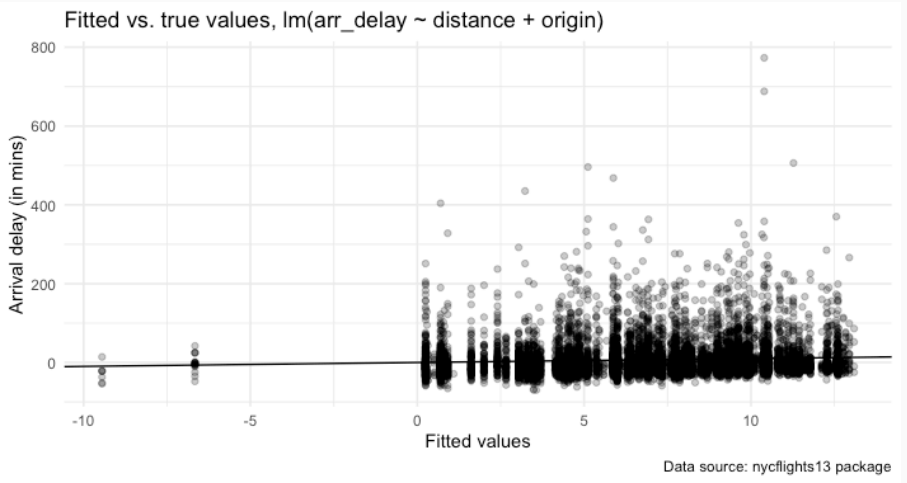
Standardization (1 unit = 1 s.d. on the covariate)

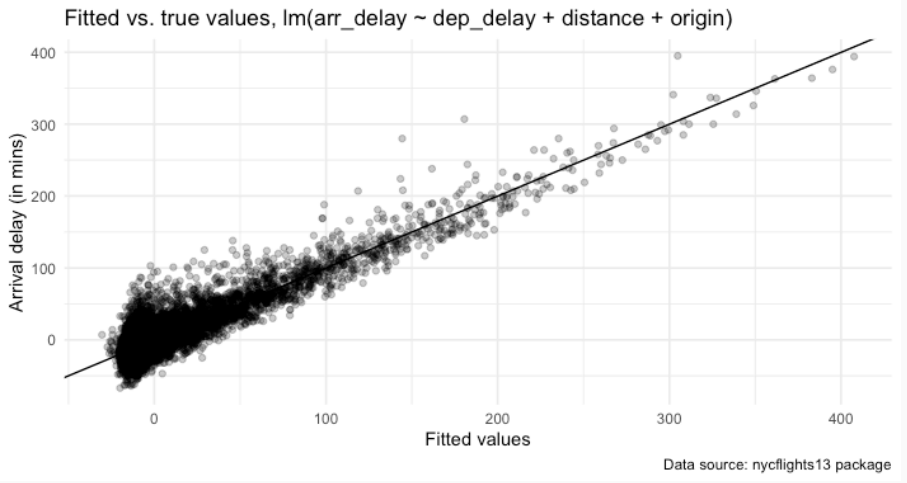
```
R> # rescale continuous variable
R> flights$distance_std <- standardize(flights$dista
R> model_out_std <- lm(arr_delay ~
+        distance_std + origin, data = flights)
R>
R> # plot model (detailed fine-tuning not shown)
R> modelplot(model_out_std)
```



More reporting with plots

- There is much, much more that can be done with plots and model reporting (stay tuned for the session on visualization!).
- Always be aware about what a model gives you and which relationships you want to explore or highlight.
- As another example, see the fitted vs. true plots on the right. The upper scatter plot compares fitted (x-axis) vs. true (y-axis) values from our standard model. It illustrates a really poor fit.
- The lower scatter plot compares fitted-vs-true for a slight modification of our standard model in which we also take `dep_delay` into account as predictor. Apparently, its a very powerful one.





Fitting many models

Decisions in the modeling workflow

Big data and the need for models

- In the early days of the big data hype, people were overly enthusiastic about its implications for modeling (see quote on the right).
- This is falling for the inception that we can simply "let the data speak".
- However, the data science workflow is a sequence of subjective decisions from start to finish, with lots of researcher degrees of freedom.
- Think of all the weakly justified decisions regarding:
  - data collection / selection
  - measurement
  - model choice
  - model specification
  - reporting

> "Scientists are trained to recognize that correlation is not causation, that no conclusions should be drawn simply on the basis of correlation between X and Y. (...) Once you have a model, you can connect the data sets with confidence. Data without a model is just noise. (...) There is now a better way. Petabytes allow us to say: "Correlation is enough." We can stop looking for models. We can analyze the data without hypotheses about what it might show."

Chris Anderson, "The End of Theory: The Data Deluge makes the Scientific Method Obsolete (2008, *Wired*)"

Decisions in the modeling workflow

Big data and the need for models

- In the early days of the big data hype, people were overly enthusiastic about its implications for modeling (see quote on the right).
- This is falling for the inception that we can simply "let the data speak".
- However, the data science workflow is a sequence of subjective decisions from start to finish, with lots of researcher degrees of freedom.
- Think of all the weakly justified decisions regarding:
  - data collection / selection
  - measurement
  - model choice
  - model specification
  - reporting

Question: How do researchers usually deal with this?

> "Scientists are trained to recognize that correlation is not causation, that no conclusions should be drawn simply on the basis of correlation between X and Y. (...) Once you have a model, you can connect the data sets with confidence. Data without a model is just noise. (...) There is now a better way. Petabytes allow us to say: "Correlation is enough." We can stop looking for models. We can analyze the data without hypotheses about what it might show."

Chris Anderson, "The End of Theory: The Data Deluge makes the Scientific Method Obsolete (2008, *Wired*)"

Nested models

TABLE 3 Change in Pork Earmark Spending by Zip Code from Pre- to Post-Redistricting Years

	Dependent Variable: Change in Logged Earmark Spending per Capita, per Year				
	Model (1)	**Model (2)**	**Model (3)**	**Model (4)**	**Model (5)**
Proposition 1(c):	−1.23*	−1.50**	−1.50**	−1.46**	−1.61**
Δ District Diversity Index ($D_{POST} - D_{PRE}$)	(0.55)	(0.55)	(0.55)	(0.55)	(0.57)
Per Capita Income ($1,000s)	−0.0072**	−0.0063*	−0.0057*	−0.0056*	−0.0056
	(0.0027)	(0.0028)	(0.0029)	(0.0029)	(0.0029)
Poverty Rate	−0.73	−0.78	−0.74	−0.73	−0.66
	(0.42)	(0.42)	(0.42)	(0.42)	(0.43)
Racial Minority	0.36**	0.34**	0.46*	0.52**	0.49**
	(0.12)	(0.12)	(0.19)	(0.18)	(0.19)
Population Density	0.0073	0.21	0.54	0.66	0.62
(1,000,000s/Sq. Mi.)	(1.38)	(1.38)	(1.44)	(1.43)	(1.43)
Democrat (Pre-Redistricting) to Republican (Post-Redistricting) Senator	—	0.39***	0.40***	0.40***	0.49***
		(0.099)	(0.10)	(0.099)	(0.11)
Republican (Pre-Redistricting) to Democrat (Post-Redistricting) Senator	—	−0.24	−0.25	−0.25	−0.27
		(0.18)	(0.18)	(0.18)	(0.18)
2000 Gore Vote Share	—	—	−0.30	—	—
			(0.36)		
|2000 Gore Vote Share −0.50|	—	—	—	−0.57	−0.41
				(0.44)	(0.44)
Post-Redistricting Senator is More Junior	—	—	—	—	−0.039
					(0.079)
Post-Redistricting Senator is More Senior	—	—	—	—	−0.17
					(0.087)
Constant	0.078	0.046	0.16	0.047	0.090
	(0.098)	(0.099)	(0.18)	(0.099)	(0.11)
N	1,599	1,599	1,599	1,599	1,599

***p < .001; **p < .01; *p < .05; (two-tailed); standard errors in parentheses.
The dependent variable is measured as $\log(Y_Z^{POST}/Population_Z + 1) - \log(Y_Z^{PRE}/Population_Z + 1)$, where Y_Z^{POST} represents per-year pork spending in zip code Z during 2003–2004, and Y_Z^{PRE} is the same measurement for years 1998–2002.

`Credit` Chen 2010, The Effect of Electoral Geography on Pork Barreling in Bicameral Legislatures

TABLE 3. HIERARCHICAL REGRESSION ANALYSIS OF PREDICTORS OF CIVIC PARTICIPATION

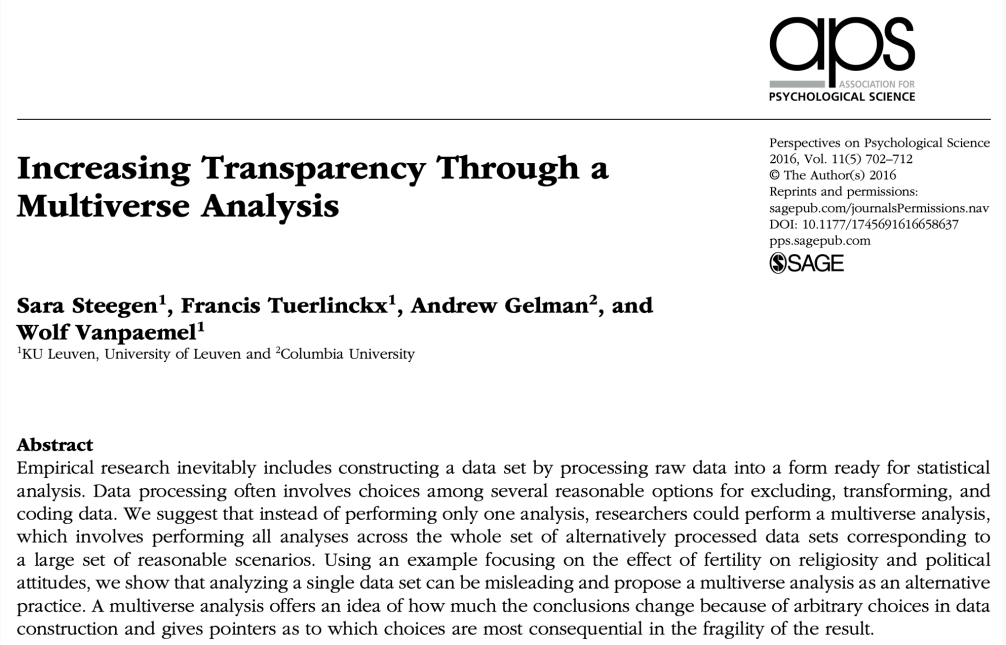
Predictor variables	*Regression 1*	*Regression 2*	*Regression 3*
Gender (1, female; 0, male)	0.06***	0.05***	0.05***
Hometown (1, Texas; 0, elsewhere)	−0.04**	−0.03*	−0.03*
Ethnicity			
Black (1, yes; 0, no)	−0.04	−0.04	−0.05
Latino (1, yes; 0, no)	0.02	0.02	0.01
White (1, yes; 0, no)	0.01	−0.00	0.00
Year in school (1, freshman; 6, doctoral)	−0.21**	−0.21***	−0.21*
Parents' education (1, less than high school; 5, graduate)	0.02***	0.02***	0.02**
Life satisfaction		0.27***	0.25***
Social trust		0.23***	0.20***
Needs for using Groups			
Socializing			−0.01
Entertainment			−0.10
Self-status seeking			0.01
Information			0.14***
R^2	0.04	0.08	0.16
R^2 change	0.04	0.05	0.08

*$p < 0.05$; **$p < 0.01$; ***$p < 0.001$.

`Credit` Park et al. 2009, Being immersed in social networking environment: Facebook groups, uses and gratifications, and social outcomes

Exploring the model space

- **Another idea:** run not an arbitrary (small) set of models but as many (plausible ones) as possible to get an idea how much conclusions change depending on arbitrary data wrangling and modeling choices (the "model distribution").
- There are various related procedures and labels used in different subfields to promote this idea, including:
  - Multiverse analysis (Steegen et al. 2016)
  - Specification curves (Simonsohn et al. 2020)
  - Computational multimodel analysis (Young and Holsteen 2015)
- Also check out critical perspective on "Mülltiverse Analysis" (Julia Rohrer). The bottom line: Mindless multiversing doesn't give more robustness or insight.



aps ASSOCIATION FOR PSYCHOLOGICAL SCIENCE

Increasing Transparency Through a Multiverse Analysis

Perspectives on Psychological Science
2016, Vol. 11(5) 702–712
© The Author(s) 2016
Reprints and permissions:
sagepub.com/journalsPermissions.nav
DOI: 10.1177/1745691616658637
pps.sagepub.com
SAGE

Sara Steegen[1], Francis Tuerlinckx[1], Andrew Gelman[2], and Wolf Vanpaemel[1]
[1]KU Leuven, University of Leuven and [2]Columbia University

Abstract
Empirical research inevitably includes constructing a data set by processing raw data into a form ready for statistical analysis. Data processing often involves choices among several reasonable options for excluding, transforming, and coding data. We suggest that instead of performing only one analysis, researchers could perform a multiverse analysis, which involves performing all analyses across the whole set of alternatively processed data sets corresponding to a large set of reasonable scenarios. Using an example focusing on the effect of fertility on religiosity and political attitudes, we show that analyzing a single data set can be misleading and propose a multiverse analysis as an alternative practice. A multiverse analysis offers an idea of how much the conclusions change because of arbitrary choices in data construction and gives pointers as to which choices are most consequential in the fragility of the result.

Specifying multiple models

When one model is not enough

- Often we want to specify not one but multiple models. They can differ in model family, outcome, covariate set, transformations, or data.
- If outcomes/features vary, so does the formula. Treating formulas as strings, we can generate them programmatically:

```
R> xvars <- paste0("x", 1:10)
R> as.formula(paste("y ~", paste(xvars, collapse = "
```

```
y ~ x1 + x2 + x3 + x4 + x5 + x6 + x7 + x8 + x9 + x10
```

Example: extreme bounds analysis

- Extreme bounds analysis (EBA) estimates models with *all* combinations of a set of covariates and looks at the distribution (or range: the extreme bounds) of a coefficient across them.
- Which determinants are robustly associated with the outcome, whatever else is in the model?
- There are packages for this (`ExtremeBounds`), but the basics are easy to build ourselves.

Step 1: Define outcome and covariate set

```
R> depvar <- "arr_delay"
R> covars <- c("dep_delay", "carrier", "origin",
+              "air_time", "distance", "hour")
```

That's 63 possible non-empty subsets of covariates, hence 63 models.

Specifying multiple models (cont.)

Step 2: Build function to run lm models across set of all possible variable combinations

```
R> combn_models <- function(depvar, covars, data)
+ {
+   combn_list <- list()
+   # generate list of covariate combinations
+   for (i in seq_along(covars)) {
+     combn_list[[i]] <- combn(covars, i, simplify = FALSE)
+   }
+   combn_list <- unlist(combn_list, recursive = FALSE)
+   # function to generate formulas
+   gen_formula <- function(covars, depvar) {
+     form <- as.formula(paste0(depvar, " ~ ", paste0(covars, collapse = "+")))
+     form
+   }
+   # generate formulas
+   formulas_list <- purrr::map(combn_list, gen_formula, depvar = depvar)
+   # run models
+   models_list <- purrr::map(formulas_list, lm, data = data)
+   models_list
+ }
```

Specifying multiple models (cont.)

Step 3: Run the models (on a 10,000-flight sample; the full data makes a heavy list)

```
R> models_list <- combn_models(depvar = depvar,  covars = covars, data = flights)
```

How many models did we fit?

```
R> length(models_list)
```

```
  [1] 63
```

And what did we get? A glimpse at the first list element:

```
R> models_list[[1]]
```

```

  Call:
  .f(formula = .x[[i]], data = ..1)

  Coefficients:
  (Intercept)    dep_delay
       -5.836        1.011
```

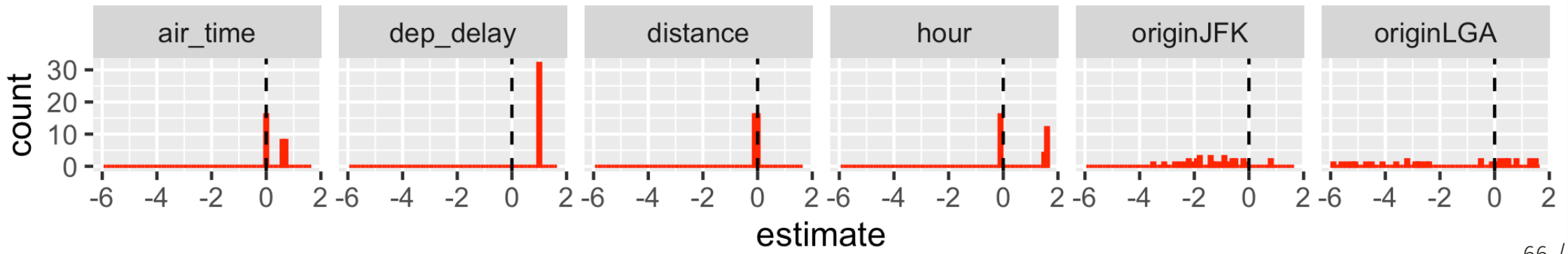
Evaluating many models with broom

Step 4: Extract tidy results from all models and stack them (this is where `broom` shines)

```
R> models_broom_df <- map(models_list, tidy) |> list_rbind(names_to = "model")
```

Step 5: Summarize the estimates for key predictors

```
R> models_broom_df |>
+   filter(!str_detect(term, "Intercept|carrier")) |>
+   ggplot(aes(estimate)) +
+   geom_histogram(binwidth = .1, color = "red") +
+   geom_vline(xintercept = 0, linetype = "dashed") +
+   facet_grid(cols = vars(term), scales = "free_y")
```



Specification curve analysis

- **Specification curve analysis** (SCA; Simonsohn, Simmons and Nelson 2020) formalizes the idea: instead of reporting one (or a few) arbitrarily chosen specifications, report *all reasonable* ones.
- Three main steps:
  1. **Define** the set of reasonable specifications;
  2. **Estimate** all of them and report the results in a *descriptive specification curve*;
  3. **Test** the results jointly using an *inferential specification curve*.
- What counts as "reasonable"? A specification should be **theoretically justified** (a sensible test of the hypothesis), **statistically valid** (not obviously misspecified), and **non-redundant** with the others in the set.

nature human behaviour

RESOURCE

https://doi.org/10.1038/s41562-020-0912-z

Check for updates

Specification curve analysis

Uri Simonsohn[1]✉, Joseph P. Simmons[2] and Leif D. Nelson[3]

Empirical results hinge on analytical decisions that are defensible, arbitrary and motivated. These decisions probably introduce bias (towards the narrative put forward by the authors), and they certainly involve variability not reflected by standard errors. To address this source of noise and bias, we introduce specification curve analysis, which consists of three steps: (1) identifying the set of theoretically justified, statistically valid and non-redundant specifications; (2) displaying the results graphically, allowing readers to identify consequential specifications decisions; and (3) conducting joint inference across all specifications. We illustrate the use of this technique by applying it to three findings from two different papers, one investigating discrimination based on distinctively Black names, the other investigating the effect of assigning female versus male names to hurricanes. Specification curve analysis reveals that one finding is robust, one is weak and one is not robust at all.

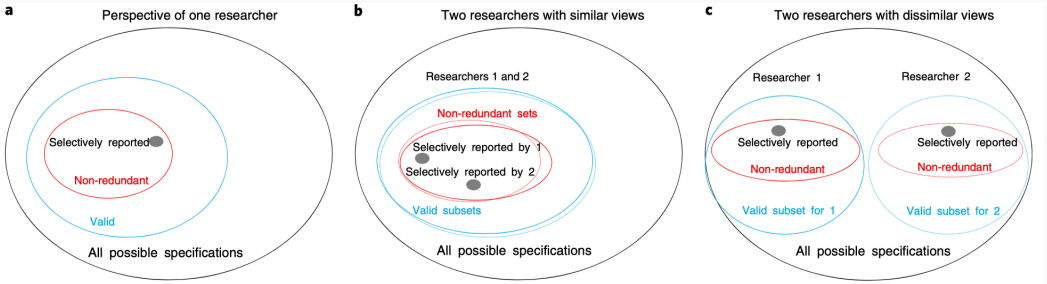


Fig. 1 | Sets of possible specifications as perceived by researchers. a, The set of specifications reported in an article are a small subset of those the researcher would consider valid to report. **b**, Different researchers may have similar views on the set of valid specifications but report quite different subsets of them. **c**, Different researchers may also disagree on the set of specifications they consider valid.

Specification curve analysis (cont.)

The descriptive curve

- All estimates sorted by effect size, significance highlighted, plus a dashboard underneath showing which decisions went into each specification.
- Reveals *which* decisions drive the results, e.g., whether an effect only appears when a certain control is excluded.

The inferential curve

- How extreme is the observed curve if the true effect were zero?
- Simulate the null by resampling with the effect of interest removed, re-run all specifications on each resample, and compare.
- Three joint test statistics: **median effect size**, **share of significant specifications**, **aggregate z** (Stouffer).

In practice

- Example in the paper: are hurricanes with female names deadlier? Across 1,728 reasonable specifications, only about a third of the estimates are significant; the joint tests fail to reject the null.
- R packages: `specr` (see here) and `multiverse` (see here).
- The building blocks are what we just did: generate formulas programmatically, fit models with `map()`, collect results with `broom`.
- Caveat: SCA does not free you from thinking. The set of specifications has to be defined *ex ante* and justified. Padded with implausible models, the curve is meaningless ("Mülltiverse").

Specification curves in the wild

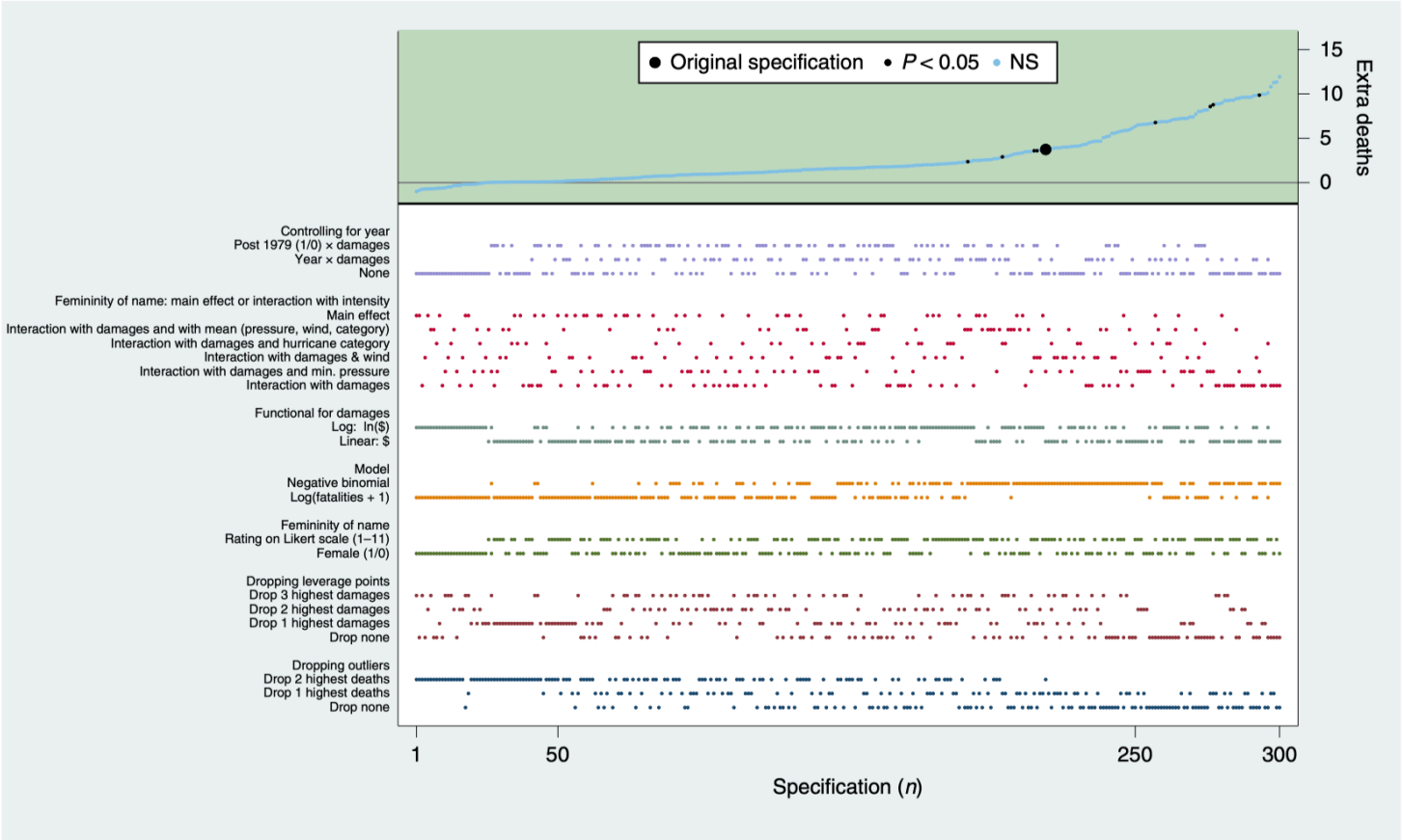


Fig. 2 | Descriptive specification curve. Each dot in the top panel (green area) depicts the marginal effect, estimated at sample means, of a hurricane having a female rather than male name; the dots vertically aligned below (white area) indicate the analytical decisions behind those estimates. A total of 1,728 specifications were estimated; to facilitate visual inspection, the figure depicts the 50 highest and lowest point estimates and a random subset of 200 additional ones, but the inferential statistics for specification curve analysis include all 1,728 specifications. NS, not significant.

Summary

Some final bits of advice that didn't fit on the main slides

1. Before you actually run models, **describe the data**. That's not only for you. Every report should begin with a visualization of the phenomena of interest, plotting the rawest data available that is also legible in a graph.

2. **Plot early, and plot often**! Visualization is not only a great tool for communication, but also for exploration and statistical analysis (more on that later).

3. **Fit many models**. At least if you don't commit to one particular subset (→ preregistration). Think in terms of series of models, starting with the utterly simple and continuing through to the hopelessly complex.

4. **Never judge a model on the data it was trained on**. Split first, resample to compare and tune, touch the test set once.

5. **Table results with care**. Please don’t just report which effects were significant and which were non-significant. Please don’t report just p-values. Please don’t just report the estimated effects of the significant effects.

6. **Invest more time in refining plots** than you planned to. Good figures aren't only plots. Think of informative headers and notes. Ideally, your figures are self-contained.